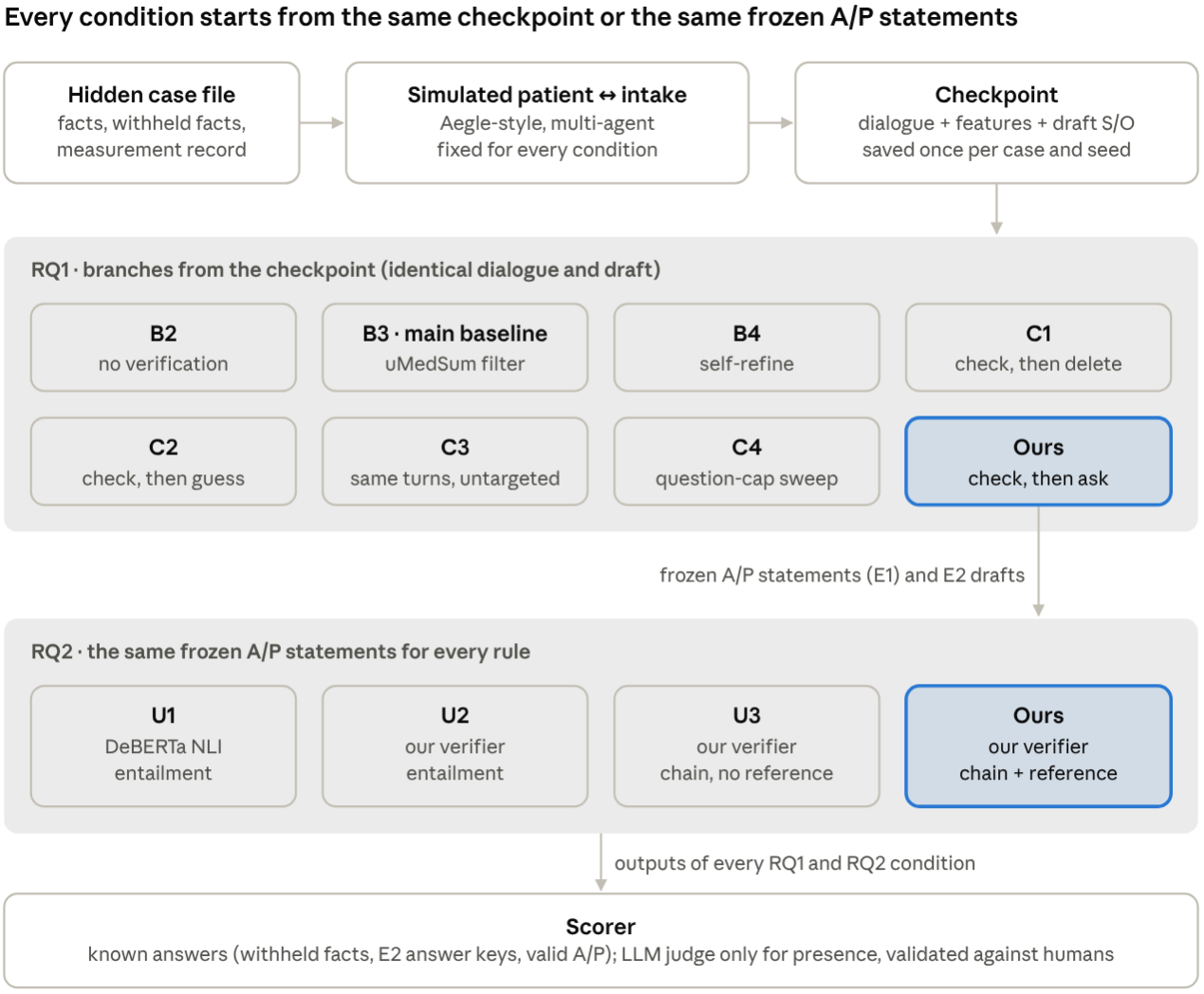
One Encounter, Two Readers — Implementation Plan (Oct 13–31)

Oct 13, 2026 · @Nandhu

Overview

This is the working plan for the final proposal, *One Encounter, Two Readers*, from **Tue Oct 13 to Sat Oct 31**. We test two things: whether asking the patient when verification finds a gap beats deleting, guessing or asking untargeted questions (RQ1), and whether checking diagnoses and plans by their evidence chain keeps correct reasoning that entailment filters delete (RQ2). Everything else is a fixed substrate or a supporting component with one check.



Experiment structure · RQ1 conditions branch from one checkpoint; RQ2 rules share one set of A/P statements

Comparisons are fair by construction: every RQ1 condition continues from the identical saved dialogue and draft, and every RQ2 rule judges the identical A/P statements, so any difference comes from what the condition does, not from a different conversation.

The TA's reply on the proposal is expected by Oct 15. The plan starts today on work that every version of the project needs, uses only dev data until the design is final, and adjusts at the feedback gate.

Start now vs. wait for feedback

Start today on everything that **any** accepted version of the proposal needs; hold back only what depends on the final list of conditions. Every item in the left-hand table is reused whatever the TA says, so no work is lost if the design changes.

One hard rule before the gate: run nothing on the test set. Until the design is frozen at the feedback gate, every run uses the 10 **dev** encounters (ACI-Bench valid). The 40 test encounters (ACI-Bench test1) stay sealed until Oct 21, so no prompt, threshold or rule is ever tuned on the data we report.

Safe to start now (Oct 13–15)

Work	Owner	Why it is safe
Environment, repository, OpenRouter account with a spending cap, budget ledger	Surya	Needed by every version
QLoRA dry run on one 1080 Ti (timing and memory)	Namrata	Decides where the verifier is trained; needed even if fine-tuning is later reduced
Download ACI-Bench and MTS-Dialog; freeze splits	Rahul, Jyothika	Data does not depend on the conditions
Case files: atomic facts, Subjective/Objective tags, withheld facts, measurement record	Jyothika (all review)	The known answers every metric needs
Simulated patient and its fidelity audit on dev	Surya	Every encounter-mode condition uses it
Aegle-style intake and checkpoint save/load	Surya	The fixed substrate shared by all RQ1 conditions
Turn retrieval (BM25 + dense + rank fusion), measurement-record tool	Rahul	Used by the verifier in every version

Work	Owner	Why it is safe
DeBERTa NLI wrapper and the uMedSum filter (B3)	Rahul	B3 is the main baseline; DeBERTa is also U1 and S1
Verifier training-data builder; training run 1 once the dry run passes	Namrata	Small compute; useful even if fine-tuning is cut
Planted-error notes (E2) with answer keys	Jyothika	Ground truth for RQ2 and the verifier
Known-answer metric code: withheld-fact recovery, fact recall, valid-claim retention	Jyothika	Metrics follow directly from the proposal
Experiment 1 pilot on dev: B2 → B3 and U1 on 10 dev encounters	Jyothika, Rahul	Measures the problem itself; gives early evidence to show the TA

Wait for the feedback gate

Work	Why it waits
Final run matrix: conditions, seeds, sample sizes	The TA may add or cut conditions
Running controls C1–C4 and RQ2 conditions U1–U3 on test data	Their code paths can be written now and tested on dev, but test runs wait for the frozen design
Unsealing the 40 test encounters	Only after the design and prompts are frozen (Oct 21)
Annotation and report writing beyond the proposal text	Depend on the final design and results

Feedback gate

Feedback is expected on **Oct 14 or 15**. If none has arrived by **Fri Oct 16, 12:00**, we proceed with the proposal as submitted, and any later feedback is applied as an addition, not a restart.

Procedure on the day feedback arrives

1. Within 2 hours, Jyothika copies every point into `docs/ta_feedback.md`, one line per point.
2. Same day, a 30-minute call where each point is sorted into one of three classes:
   - **Wording:** changes to framing or definitions only. Update the report draft; no plan change.
   - **Additive:** a new condition, metric or check. Add it to the run matrix and pay for it from the cut list, so the end date does not move.
   - **Structural:** a change to a research question or the core method. Re-plan the build phase that evening; the start-now work is still reused.
3. Update this document the same evening: the run matrix, any affected sections, and a dated change note at the top of the scenario table below.

Likely feedback and what it changes

If the TA says…	We change	Work done so far that stays valid
Accepted as is	Nothing; the build phase starts Oct 16	All
Too many conditions	Cap sweep C4 reduced to caps 1 and 3; drop S3 and B4; keep B3, C1–C3 and U1–U3	All
The simulated patient is not realistic	Report the fidelity audit numbers; add 5 encounters in which a team member plays the patient from the case file, as a sanity check	All; case files double as role-play scripts
Withheld facts are artificial	Also report *natural* gaps: facts missing at the checkpoint without withholding (already computed in Experiment 1)	All
Add a stronger baseline	Add B5: a larger model via OpenRouter as a single-agent doctor; about US$5–10 more	All
Show that RQ2 matters clinically	20 A/P statements rated by a medical student or clinician, if one is available	All

If the TA says…	We change	Work done so far that stays valid
Fine-tuning is unnecessary	Use the prompted 7B as the verifier; report S1 only as a comparison	Verifier data becomes an evaluation set
Merge or reword the research questions	Report wording only	All experiments unchanged

Team ownership

Each component has one **owner**, who builds it, and one **reviewer**, who reads all of its code and must be able to explain it in the viva. Owners follow strengths; reviewers are crossed so knowledge spreads.

Member	Strength	Owns	Reviews
Rahul	RAG	Turn retrieval (BM25 + dense + rank fusion) and tuning of k; measurement-record tool; MedlinePlus reference passages; DeBERTa NLI wrapper; **B3** (uMedSum filter); **U1**, **U3**, **S2**	Surya's controller and branch runner
Surya	RAG, agents	Repository and OpenRouter client; simulated patient; Aegle-style intake; checkpoint save and branch runner; tools and guards; **controller (verify-then-ask)**; **B1**, **B2**, **C1–C4**, **S3**; run scripts; demo	Rahul's retrieval and B3
Jyothika	Math, analysis	Splits; case files, withheld facts, measurement records, red flags; **E2** planted-error notes; all metrics; LLM-judge validation; statistics (bootstrap, McNemar, Holm, κ); annotation protocol; **Experiment 1**; tables, figures, failure analysis	Namrata's verifier evaluation

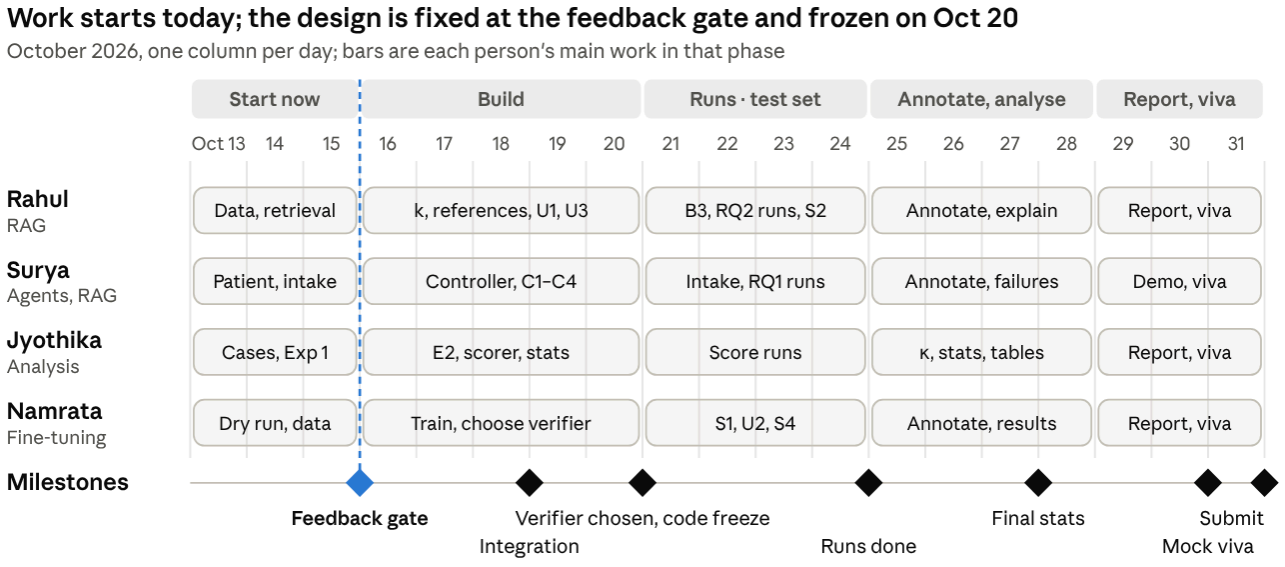
Member	Strength	Owns	Reviews
Namrata	Fine-tuning	Verifier dry run, training data, QLoRA training; verifier inference with label confidence; S/O and A/P prompt modes; **U2**, **S1**; patient summary and its check; **S4**; **B4** (self-refine)	Jyothika's metric code

Integration: Surya merges components into the full pipeline; Rahul is the second integrator, so the system never depends on one person.

Shared by all four: reviewing case files (Oct 14 and Oct 17), annotation in pairs (Oct 25–26), the report, the demo rehearsal and the mock viva.

Timeline

Nineteen days, four parallel tracks. The dashed line is the feedback gate: before it, everything runs on dev data and is reusable whatever the TA says; after it, the design is fixed, and it is frozen on the evening of Oct 20.



Schedule · Oct 13–31, four tracks, five phases, seven milestones

Surya's build phase is the critical path: the controller needs Rahul's retrieval, Namrata's verifier (chosen Oct 20) and Jyothika's scorer before the freeze. Rahul is the second integrator if Surya falls behind.

All four together: kickoff Oct 13; dev case-file review Oct 14; test case-file review Oct 17; integration day Oct 18; cross-training Oct 20; annotation Oct 25–26; results session Oct 29; mock viva Oct 30; submission Oct 31.

Day-by-day tasks

Five phases: **start now** (Oct 13–15, dev only), **build** (Oct 16–20), **runs** (Oct 21–24, test set unsealed), **annotation and analysis** (Oct 25–28), **report and viva** (Oct 27–31). Each phase ends with a checkpoint; anything unticked becomes its owner's first task the next morning.

Phase 0 — Start now (Tue Oct 13 to Thu Oct 15, dev only)

Day	Rahul	Surya	Jyothika	Namrata
Tue 13	Download ACI-Bench and MTS-Dialog; set up the cluster environment; DeBERTa NLI smoke test on a 1080 Ti	30-min kickoff (all); repository skeleton; OpenRouter client with token and turn counter; first test call	Split script (dev, test, few-shot pool); case-file extraction prompt; run it on the 10 dev encounters	Install bitsandbytes (CUDA 12.1–12.6 build); QLoRA dry run, 3B, 200 steps; record seconds per step and peak memory
Wed 14	Turn index; BM25 + dense + rank fusion; measurement-record tool; list dev chief complaints for MedlinePlus	Simulated patient v1; Aegle-style intake v1 on dev; checkpoint save and load	**All four review the 10 dev case files** (2–3 each); apply the withheld-fact rule; machine-extract the 40 test case files (not yet reviewed)	Training-location decision from the dry run; verifier data builder (S/O and A/P examples); 100-example spot check
Thu 15	B3: uMedSum filter (NLI deletion, coverage insertion) on dev B2 notes; k-tuning harness	Simulated-patient fidelity audit on 10 dev encounters (with Jyothika); fix prompt; B2 end to end on dev, checkpoints saved	Known-answer metric code; **Experiment 1 pilot** on dev (B2, B3, U1); start E2 planted-error drafts	Start verifier training run 1 if the dry run passed

Checkpoint Oct 15: ☐ splits frozen ☐ 10 dev case files reviewed ☐ simulated-patient audit passed ☐ B2 saves checkpoints on dev ☐ retrieval returns evidence ☐ verifier training started ☐ Experiment 1 pilot numbers ☐ feedback triaged, or the default date set

Phase 1 — Build (Fri Oct 16 to Tue Oct 20, dev only)

Member	Tasks	Done when
Surya	Controller (verify-then-ask) with the S/O rules; tools and guards; branch runner (load checkpoint, run any condition); B1; C1, C2, C3, C4 configs; S3 single-agent intake	Every RQ1 condition runs on 3 dev checkpoints
Rahul	Tune k on dev (recall@1/3/5); MedlinePlus passages for all test complaints (after Oct 17 review); A/P reference retrieval; U1, U3; S2	U1 and U3 run on dev A/P statements; k fixed in config
Namrata	Training run 2 (without A/P examples); inference server returning label and confidence; U2 prompt mode; patient summary and summary check; S4; B4; S1 on verifier dev	Verifier chosen by **Tue Oct 20** and written in config
Jyothika	**All four review 10 test case files each on Sat Oct 17**; E2 complete with answer keys and a second-person check; judge prompts; statistics code; annotation guideline	E2 frozen; scorer produces a full dev table

Sun Oct 18 — integration day: every condition runs end to end on 3 dev encounters. **Tue Oct 20 — dev pilot:** every condition on all 10 dev encounters (1 seed); compute and token estimates updated; cross-training session 1 in the evening.

Checkpoint Oct 20: ☐ all conditions run on dev ☐ verifier chosen ☐ prompts frozen and committed ☐ run matrix final

Phase 2 — Runs (Wed Oct 21 to Sat Oct 24, test set unsealed)

Date	Who	Runs
Wed 21 morning	All	**Code freeze**; record the commit hash in the ledger; unseal test1
Wed 21	Surya	Multi-agent intake on 40 encounters × 2 seeds (80 checkpoints); single-agent intake for B1 and S3
Thu 22	Surya, Rahul	RQ1 branches from every checkpoint: B2, B3, B4, ours, C1, C2, then C3 (matched to ours), C4

Date	Who	Runs
Thu 22–Fri 23	Rahul, Namrata	RQ2 on E2 and on E1 A/P statements: U1, U2, U3, ours; S1 on E2
Fri 23–Sat 24	Surya, Namrata	S2, S3, S4; safety tallies; re-run failed encounters once
Ongoing	Jyothika	Score each block as it finishes; flag anomalies the same day

Checkpoint Oct 24: ☐ every block in the run matrix marked done ☐ failed encounters at most 2 per condition ☐ raw outputs backed up off the cluster

Phase 3 — Annotation and analysis (Sun Oct 25 to Wed Oct 28)

Date	Who	Task
Sun 25	All	Calibration on 3 notes together, then annotation in pairs (Rahul + Jyothika, Surya + Namrata)
Mon 26	All	Finish 30 notes and 20 summaries
Tue 27	Jyothika	κ, judge validation, final statistics with Holm correction
Mon 26–Wed 28	All	Failure analysis of 30 errors by first failing stage; each owner writes the playbook diagnostics for their comparisons

Phase 4 — Report and viva (Tue Oct 27 to Sat Oct 31)

Date	Who	Task
Tue 27–Thu 29	Each owner	Report sections for their components; Jyothika assembles tables and figures
Thu 29	All	Results session: each owner explains their results using the playbook; Surya rehearses the demo
Fri 30	All	Mock viva: each member answers questions on components they did not build
Sat 31	All	Final check and submission: report, code, run outputs, ledger, demo

Environment and compute setup

Owners: Surya (environment, OpenRouter), Namrata (GPU checks). **Due:** Tue Oct 13.

Where each model runs

Model	Runs on	Why
Qwen2.5-7B-Instruct: intake agents, note writer, simulated patient, question wording	OpenRouter API	vLLM's standard builds need compute capability 7.0+; the 1080 Ti is 6.1
Verifier: Qwen2.5-3B-Instruct + QLoRA adapter	Cluster 1080 Ti (training: see dry run below)	Small, local, runs dozens of times per encounter
DeBERTa-v3-large NLI (B3, U1, S1)	Cluster 1080 Ti, fp32	Fits easily in 11 GB
MiniLM embeddings, BM25	Cluster (GPU or CPU)	Tiny
LLM judge (evaluation only)	OpenRouter, larger model	Chosen and priced on Oct 15

Three facts about the 1080 Ti that change the configuration

1. **bitsandbytes 4-bit (NF4) works on Pascal**, but only through wheels built for CUDA 11.8–12.6; the CUDA 12.8+ builds do not include Pascal (bitsandbytes install docs). Install PyTorch from the cu121 or cu124 index.
2. **No bf16, and fp16 arithmetic is very slow on consumer Pascal cards.** Set the 4-bit compute dtype to **float32**, and turn off `bf16` and `fp16` in the trainer.
3. **No FlashAttention.** Use `attn_implementation="sdpa"`.

Setup

```
conda create -n vta python=3.11 -y && conda activate vta
pip install torch==2.4.1 --index-url https://download.pytorch.org/whl/cu121
pip install "transformers>=4.44,<5" "peft>=0.12" "trl>=0.10"
"bitsandbytes>=0.43" accelerate datasets \
            sentence-transformers rank-bm25 pysbd pydantic openai textstat
scipy pandas pyyaml tqdm pytest
nvidia-smi                      # driver must support CUDA 12.1
python -c "import torch; print(torch.cuda.get_device_name(0),
torch.cuda.get_device_capability(0))"   # expect (6, 1)
python -c "import bitsandbytes as bnb; print(bnb.__version__)"
```

OpenRouter client (src/vta/llm.py)

```
import os, time
from openai import OpenAI

client = OpenAI(base_url="https://openrouter.ai/api/v1",
api_key=os.environ["OPENROUTER_API_KEY"])
BACKBONE = "qwen/qwen-2.5-7b-instruct"

def chat(messages, temperature=0.2, max_tokens=512, ledger=None, tag=""):
    for attempt in range(4):
        try:
            r = client.chat.completions.create(model=BACKBONE,
messages=messages,
                                               temperature=temperature,
max_tokens=max_tokens)
            if ledger is not None:
                ledger.add(tag, r.usage.prompt_tokens,
r.usage.completion_tokens)
            return r.choices[0].message.content
        except Exception:
            time.sleep(2 ** attempt)
    raise RuntimeError("OpenRouter call failed 4 times")
```

Set a hard spending limit of **USD 50** in the OpenRouter console on day one. Every call goes through `chat()`, so every token is logged.

Verifier dry run (Namrata, Tue Oct 13)

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig
from peft import LoraConfig, prepare_model_for_kbit_training, get_peft_model

bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_use_double_quant=True,
bnb_4bit_compute_dtype=torch.float32)
name = "Qwen/Qwen2.5-3B-Instruct"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name, quantization_config=bnb,
                                             attn_implementation="sdpa",
device_map={"": 0})
model.gradient_checkpointing_enable()
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(r=16, lora_alpha=32,
lora_dropout=0.05, task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj",
"up_proj", "down_proj"]))
# Train 200 steps: batch 2, gradient accumulation 16, max length 512,
paged_adamw_8bit, lr 2e-4, bf16=False, fp16=False.
# Record: seconds per step, torch.cuda.max_memory_allocated().
```

Decision rule (Wed Oct 14): projected full run = steps per epoch × 2 epochs × seconds per step.

Projected time on one 1080 Ti	Decision
8 hours or less	Train on the cluster; runs 1 and 2 in parallel on two GPUs
More than 8 hours	Train on Kaggle or Colab **T4** (fp16 compute is fast there); keep the 1080 Ti for inference. Avoid Kaggle's P100, which is also Pascal
Out of memory	Max length 384, batch 1, accumulation 32; if still failing, Qwen2.5-1.5B-Instruct

Repository and shared data formats

One repository; every component reads and writes the formats below, defined once in `src/vta/schemas.py`. A format changes only after it is agreed in the team call and updated here the same day.

Directory tree (owner in brackets)

```
vta/
├── configs/      models.yaml, budgets.yaml, prompts/,
conditions/{b1,b2,b3,b4,c1,c2,c3,c4,ours,u1,u2,u3,s1..s4}.yaml   [Surya]
├── data/         raw/, splits.json, cases/, e2/, verifier/, references/
[Jyothika, Rahul]
├── src/vta/
|   ├── llm.py, schemas.py, tools.py, guards.py
[Surya]
|   ├── patient.py        simulated patient
[Surya]
|   ├── intake.py         orchestrator, specialists, aggregator, checkpoint
[Surya]
|   ├── controller.py     verify-then-ask rules, A/P rules
[Surya]
|   ├── retrieval.py      turn index, hybrid search, measurements, reference
passages                      [Rahul]
|   ├── nli.py, umedsum.py
[Rahul]
|   ├── verifier.py       inference + confidence, S/O and A/P prompt modes
[Namrata]
|   ├── summary.py        patient summary and its check
[Namrata]
|   └── eval/             metrics.py, judge.py, stats.py
[Jyothika]
├── scripts/      10_split.py, 20_cases.py, 30_e2.py, 40_train_verifier.py,
50_intake.py, 55_branch.py, 60_score.py, 70_report.py
├── docs/         ta_feedback.md, decision_log.md, viva_questions.md
└── runs/         <block>/<condition>/<case>_<seed>/ ; ledger.csv
```

Case file (hidden from every system; read only by the simulated patient and the scorer)


```
{"case_id": "aci_test1_017", "split": "test", "chief_complaint": "chest
tightness on stairs",
 "facts": [
   {"fact_id": "f01", "section": "S", "field": "HPI",        "text": "chest
pressure on stairs for 2 months", "mandatory": true,  "withheld": false,
"critical": false},
   {"fact_id": "f05", "section": "S", "field": "MEDICATION", "text":
"amlodipine 5 mg every morning",          "mandatory": true,  "withheld":
true,  "critical": true},
   {"fact_id": "f06", "section": "S", "field": "ALLERGY",    "text":
"allergic to aspirin, gets hives",         "mandatory": true,  "withheld":
true,  "critical": true},
   {"fact_id": "f08", "section": "O", "field": "VITALS",     "text": "blood
pressure 148/92",                  "mandatory": false, "withheld": false,
"critical": false}],
 "red_flag": {"present": false, "fact_id": null, "category": null},
 "reference_ap": [{"ap_id": "A1", "text": "likely stable angina", "section":
"A"}]}
```


Objective facts (`section: O`) form the **measurement record**; the patient never states them.

Checkpoint (one per case and seed; every RQ1 condition loads it)


```
{"case_id": "aci_test1_017", "seed": 11, "intake": "multi_agent",
"turns_used": 14,
 "dialogue": [{"turn": 1, "speaker": "patient", "text": "...", "fact_ids":
["f01"]}],
 "features": [{"feat_id": "F1", "section": "S", "field": "HPI", "text":
"...", "evidence": [1, 2]}],
 "draft_so": ["58-year-old man, chest pressure on stairs for 2 months",
"..."],
 "commit": "a1b2c3d"}
```


The draft S/O note is written **once**, at the checkpoint, and saved, so every branch starts from the identical draft. `fact_ids` are hidden from all systems; only the scorer reads them.

Claim record (one per claim per condition)

```
{"claim_id": "S5", "section": "S", "text": "no known drug allergies",
"evidence": [],
 "label": "unsupported", "confidence": 0.88, "action": "ask",
"question_turn": 15,
 "final_text": "allergic to aspirin (hives)", "final_evidence": [15]}
```

Allowed values — `label`: supported, contradicted, unsupported, chain_ok, chain_broken. `action`: keep, correct, ask, mark_unsure, delete, flag_doctor.

Condition config (configs/conditions/c3.yaml)

```
name: c3
starts_from: checkpoint          # or: own_intake (b1, s3), frozen_ap (u1-
u3), e2_draft
extra_turns: matched_to_ours     # c1, c2: 0; ours: up to cap; b2-b4: 0
question_policy: intake          # ours: targeted; c1, c2: none
verify_so: true
on_unsupported: delete           # ours: ask; c2: guess
ap_rule: evidence_chain          # u1, u2: entailment; u3:
evidence_chain_no_reference
verifier: selected               # s1: deberta | prompted_7b | lora_run1 |
lora_run2
retrieval: top_k                 # s2: whole_dialogue
```

Ledger row (runs/ledger.csv)

```
date,block,condition,split,case_id,seed,status,patient_turns,or_prompt_tokens
,or_completion_tokens,gpu_seconds,commit
2026-10-22,R1,ours,test,aci_test1_017,11,done,15,48210,6120,95,a1b2c3d
```

Data

Owner: Jyothika (case files, E2), Rahul (downloads). **Due:** dev case files Oct 14; test case files Oct 17; E2 Oct 19.

Sources and splits

Split	Source	Size	Used for	Status before Oct 21
Dev	ACI-Bench valid	10 encounters	All prompts, thresholds, pilots, Experiment 1 pilot	Open
Test (E1)	ACI-Bench test1	40 encounters	All reported encounter results	Case files built and reviewed; **no system runs**
E2	Reference notes of the 40 test encounters	120 drafts	RQ2, verifier comparison	Built; **no system runs**
Few-shot pool	ACI-Bench train	67	In-context examples only	Open
Verifier train / dev	MTS-Dialog train / validation + ACI-Bench train	~6k / ~600	Verifier training and selection	Open

ACI-Bench is CC BY 4.0, so sending it to OpenRouter is permitted. The split script asserts that no test text appears in any training, few-shot or dev file.

Case files (50: 10 dev + 40 test)

1. The 7B model reads each ACI-Bench dialogue and its reference note and lists atomic facts, each with section (S or O), SOAP field, and text.
2. Mark `mandatory` for chief complaint, onset, duration, severity, past history, current medication and allergy.
3. Mark `critical` for allergies, medications and red-flag symptoms.
4. **Withheld facts: exactly two per case**, both Subjective and mandatory, chosen in this order of preference: allergy, current medication, past medical history, then symptom onset or duration. These are facts a real patient often does not volunteer. If the case has no allergy or medication, the case file records “no known allergies” or “no regular medicines” as the withheld fact, so the value is still only revealed when asked.
5. **Measurement record:** every Objective fact (vitals, exam findings, test results) goes into the record returned by the `get_measurements` tool. It is never spoken by the patient.

6. **Red flags:** add one red-flag fact to 8 test cases, spread over cardiac, neurological, mental-health, allergic, bleeding and sepsis categories.
7. **Manual review:** each fact is checked against the dialogue by a second person; unsupported facts are deleted and missed ones added. Target 8–25 facts per case. Dev review is Wed Oct 14; test review is Sat Oct 17, 10 files per person.

E2 planted-error notes (120)

Three drafts per test reference note (40 × 3). Of these, 90 get exactly one planted error, 15 per type, and 30 stay clean.

Error type	Section	Example	Correct action
Fabricated symptom	S	adds “reports night sweats”	ask or delete
Negation flip	S/O	“denies chest pain” → “reports chest pain”	correct
Medication or dose	S	500 mg → 1000 mg	correct, or ask if the value is absent
Laterality or duration	S/O	right → left; 3 weeks → 3 months	correct
Unsupported diagnosis	A	adds a diagnosis no fact supports	reject (flag)
Plan without evidence	P	adds a test or drug nothing justifies	reject (flag)

Each draft has an answer key: sentence id, error type, original text, planted text. A second member confirms every planted error is unambiguous.

Valid A/P set: the reference A/P statements of each of the 40 notes, split into atomic statements and counted **once per note**; these measure valid-claim retention. The 30 planted A/P errors measure rejection precision and recall.

Download

```
mkdir -p data/raw && cd data/raw
# ACI-Bench: download aci-bench-corpus.zip from figshare (article 22494601),
then:
unzip aci-bench-corpus.zip -d aci-bench
git clone https://github.com/abachaa/MTS-Dialog mts-dialog
```

Simulated patient and the intake checkpoint

Owner: Surya. **Reviewer:** Rahul. **Due:** patient and intake on dev Oct 14; audit passed Oct 15; branch runner Oct 18.

Simulated patient prompt (configs/prompts/patient.txt)

```
You are a patient in a first clinic visit. Answer only from your private case
file below.

CASE FILE (numbered facts)
{subjective_facts}

WITHHELD FACTS
{withheld_facts}

RULES
1. Never invent symptoms, history, medicines, numbers or results. If
something is not in the case file,
   say you are not sure or that you do not have it.
2. Answer only what is asked. To an open question ("anything else?"), share
at most one more fact.
3. A WITHHELD fact is revealed ONLY if the question asks directly about its
topic
   (for example, "Are you allergic to any medicines?" reveals an allergy).
Never volunteer it,
   and never reveal it to a general question such as "Anything else I should
know?".
4. Use everyday words. Do not diagnose yourself or suggest treatments.
5. {red_flag_rule}

Return JSON only: {"text": "<reply>", "fact_ids": ["f01", ...],
"said_unsure": true|false}
```

The patient sees only Subjective facts; Objective facts are never in its prompt. Settings: temperature 0.1, max 160 tokens, JSON retried twice on parse failure.

Fidelity audit (Thu Oct 15, Surya and Jyothika)

Run the intake on the 10 dev encounters and read every patient turn.

Check	Pass criterion
Invented facts	0

Check	Pass criterion
Withheld fact revealed to a non-direct question	2 or fewer in 20 withheld facts
`fact_ids` correct	95% or more of turns
Medical jargon from the patient	Rare; noted, not blocking

If a check fails, revise the prompt and re-run. Freeze the prompt once all checks pass; its commit hash goes into every checkpoint.

Intake (fixed substrate)

Aegle-style, using the prompt designs in the Aegle paper's appendix: an orchestrator selects up to 3 specialists, or none for simple follow-ups; specialists propose questions and feature updates in parallel, without seeing each other; the aggregator writes features with their evidence turns first, then asks one plain-language question. The red-flag check runs on every patient answer.

Checkpoint rule: stop when all mandatory Subjective fields are filled, or at turn 17 of the 20-turn cap. Then write the draft S/O note once (temperature 0) and save the checkpoint.

Branch runner (scripts/55_branch.py)

```
def run_branch(ckpt_path, cond_cfg, seed):
    ckpt = load_json(ckpt_path)                    # identical start for
every condition
    patient = SimulatedPatient(case=load_case(ckpt["case_id"]), seed=seed,
                               history=ckpt["dialogue"])  # resumes the same
conversation
    state = State.from_checkpoint(ckpt)
    budget = turns_allowed(cond_cfg, ckpt)          # 0 for b2-b4 and c1-c2;
up to cap for ours/c4;
                                                    # c3 = turns ours used on
this case and seed
    out = CONDITIONS[cond_cfg["name"]](state, patient, budget, cond_cfg)
    save_outputs(out, cond_cfg["name"], ckpt["case_id"], seed)
    ledger.add_row(...)
```

Run order on each checkpoint: B2, B3, B4, ours, C1, C2, **then C3**, because C3 needs the number of questions ours asked on that case and seed. C4 runs ours with caps 1, 2, 3 and 5.

Verifier

Owner: Namrata. **Reviewer:** Jyothika. **Due:** data Oct 14; run 1 started Oct 15; verifier chosen Tue Oct 20.

There is **one verifier** for the whole project: Qwen2.5-3B-Instruct with a QLoRA adapter. It only labels claims; the controller's fixed rules decide what to do with each label. The same weights serve three prompt modes.

Mode	Question it answers	Evidence in the prompt	Labels
S/O	Did the patient say this, or does the measurement record show it?	Top-k patient turns (S) or measurement entries (O)	supported, contradicted, unsupported
A/P	Do the cited facts justify this conclusion?	Cited features, plus a reference passage if found	chain_ok, chain_broken
Summary	Is this summary sentence in the signed note?	Signed-note sentences (S/O prompt format)	supported, contradicted, unsupported

Training data (~6k train, ~600 dev)

1. Split MTS-Dialog train and ACI-Bench train notes into sentences, then into atomic claims with the 7B model.
2. Pair each claim with its top-3 turns from Rahul's retriever.
3. **supported:** keep the pair if DeBERTa entailment is 0.5 or more, or the 7B model says supported; discard it if they disagree.
4. **contradicted:** perturb supported claims by negation flip, number or dose change, laterality swap, or duration change, keeping the same evidence.
5. **unsupported:** invent claims about symptoms that are absent from the dialogue, and pair real claims with unrelated turns.
6. **chain_ok / chain_broken:** reference A/P sentences paired with their supporting facts, and with facts from a different case.
7. **Spot check:** 100 random examples by hand. If fewer than 90 are correct, fix the rule that produced the errors and rebuild.

Label	Train	Dev
supported	2,000	200

Label	Train	Dev
contradicted	1,500	150
unsupported	1,500	150
chain_ok	500	50
chain_broken	500	50

Run 1 uses all examples; run 2 drops the A/P examples, to test whether A/P data helps or hurts S/O accuracy.

Prompt formats

```
S/O MODE                                                A/P MODE
SECTION: S                                              SECTION: A
CLAIM: takes metformin 500 mg twice daily               CLAIM: likely stable angina
EVIDENCE:                                               FEATURES:
[T9] DOCTOR: Any regular medicines?                     F1 chest pressure on
exertion, 2 months
     PATIENT: The metformin, the 500 one.               F2 relieved by rest in 5
minutes
[T4] PATIENT: My sugar has been okay.                   F3 current smoker
LABELS: supported | contradicted | unsupported         REFERENCE: <MedlinePlus
passage, or "none">
                                                        LABELS: chain_ok |
chain_broken
Answer: {"label": "...", "evidence": [...]}             Answer: {"label": "..."}
```

Training configuration

```
base_model: Qwen/Qwen2.5-3B-Instruct
quantization: {load_in_4bit: true, quant_type: nf4, double_quant: true,
compute_dtype: float32}  # Pascal: no bf16, slow fp16
lora: {r: 16, alpha: 32, dropout: 0.05, target_modules: [q_proj, k_proj,
v_proj, o_proj, gate_proj, up_proj, down_proj]}
max_length: 512
epochs: 2
learning_rate: 2.0e-4
scheduler: cosine
warmup_ratio: 0.03
batch_size: 2
gradient_accumulation: 16
optimizer: paged_adamw_8bit
loss_on_answer_only: true
eval_every_steps: 100        # dev macro-F1; keep the best checkpoint
seed: 2026
# On a T4 (Kaggle or Colab) instead: compute_dtype float16, fp16: true
```

Inference and confidence (src/vta/verifier.py)

The label is chosen by scoring every allowed label and taking the most likely, rather than by free generation. This gives a valid label every time and a calibrated-looking confidence that the controller's 0.8 rule uses.

```
@torch.no_grad()
def classify(prompt, labels):
    scores = []
    for lab in labels:                                    # e.g. ["supported",
"contradicted", "unsupported"]
        text = prompt + '{"label": "' + lab + '"'
        ids = tok(text, return_tensors="pt").input_ids.to(model.device)
        n = len(tok(lab, add_special_tokens=False).input_ids)
        logits = model(ids).logits[0, -n - 2:-1]          # positions
predicting the label tokens
        lp = torch.log_softmax(logits.float(), -1)
        target = ids[0, -n - 1:]
        scores.append(lp[torch.arange(len(target)), target].sum())
    probs = torch.softmax(torch.stack(scores), 0)
    i = int(probs.argmax())
    return labels[i], float(probs[i])
```

Check the slicing on 5 examples by printing the decoded target tokens before trusting it. Batch all claims of one checkpoint per call to save time.

Selection and S1

On Tue Oct 20, compare on verifier dev: DeBERTa NLI (uMedSum thresholds), the prompted 7B, LoRA run 1 and LoRA run 2. Report macro-F1, recall per error type and seconds per claim. The best dev macro-F1 becomes the verifier and is written into `configs/models.yaml` before the freeze. S1 repeats the comparison on E2 after the test set is unsealed.

Namrata must be able to explain

- What LoRA trains, and why rank 16 is enough here
- What NF4 and double quantization do, and why compute runs in fp32 on the 1080 Ti
- Why the loss is computed only on the answer tokens
- Why label scoring gives a confidence, and what it does not guarantee
- Why synthetic perturbations can flatter the verifier on E2, which is built the same way

Retrieval and reference passages

Owner: Rahul. **Reviewer:** Surya. **Due:** turn retrieval and measurement tool Oct 14; k fixed Oct 19; reference passages Oct 19.

Retrieval is a **supporting** component: it gives the verifier the right evidence for each claim, and gives the doctor a citation. We claim only what S2 and U3 show.

Turn index

1. **Unit:** one patient turn, prefixed with the doctor question just before it, stored with its turn number.
2. **Growing index:** in encounter mode the index grows after every patient answer; in E2 it is built once from the ACI-Bench dialogue.
3. **BM25 tokenisation:** lowercase; keep numbers and units; **keep negation words** (no, not, denies, never, without); no stemming of drug names.
4. **Dense:** all-MiniLM-L6-v2, cosine similarity.

Hybrid search with reciprocal rank fusion

```
def hybrid_search(query, index, k=3, rrf_k=60, depth=20):
    bm25_ranked = index.bm25_top(query, depth)       # turn ids, best first
    dense_ranked = index.dense_top(query, depth)
    score = {}
    for ranked in (bm25_ranked, dense_ranked):
        for rank, tid in enumerate(ranked, start=1):
            score[tid] = score.get(tid, 0.0) + 1.0 / (rrf_k + rank)
    return sorted(score, key=score.get, reverse=True)[:k]
```

Rank fusion combines the two rankings without rescaling BM25 and cosine scores, which live on different scales.

Choosing k (no annotation needed)

Gold evidence comes for free: the simulated patient tags each turn with the case facts it revealed, so for any claim matched to a case fact, the gold evidence turns are known. On the 10 dev encounters, compute recall@1, @3 and @5 for BM25 alone, dense alone and hybrid. Choose the **smallest k with recall of at least 0.9** and record the table for the report.

Measurement record tool

`get_measurements()` returns the encounter's Objective facts (vitals, exam, results) as numbered entries, M1, M2, and so on. Objective claims are verified against these entries, not against the dialogue. A missing Objective value is flagged for the doctor, never asked of the patient.

Reference passages (MedlinePlus)

1. After the dev and test case files are reviewed, list every chief complaint and diagnosis area (expect 25–40 topics).
2. Save the matching MedlinePlus health-topic page for each; record URL and access date in `data/references/sources.csv`.
3. Split pages by section into passages of 150–300 words, with topic and section metadata.
4. **Query:** the A/P statement plus the chief complaint, using the same hybrid search; keep the top passage only if its dense similarity is at least 0.5, otherwise pass “none”. Tune that threshold on dev.
5. In the report, call these **reference passages**, not clinical guidelines; MedlinePlus is consumer-level, public-domain health information.

Also owned by Rahul

- **DeBERTa NLI wrapper:** `MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli`, fp32, batched; returns entailment, neutral and contradiction probabilities. Used by B3, U1 and S1.
- **B3 and U1** are specified under Conditions to implement.

Rahul must be able to explain

- Why turns are the unit, and why the doctor question is prepended
- Why negation words must stay in BM25, and what breaks without them
- What reciprocal rank fusion does, and why it avoids score normalisation
- How the gold evidence for recall@k is obtained without annotation
- How uMedSum decides that information is missing, and why it misses facts that were never said

Conditions to implement

Every condition is a YAML config run by the same scripts; no condition gets its own copy of shared code. All use the same backbone, simulated patient, verifier (where one is used) and budgets.

RQ1 — all start from the same saved checkpoint

ID	Owner	Extra patient turns	After the checkpoint
B2	Surya	0	Freeze, A/P synthesis, note. No verification
B3	Rahul	0	B2's note, then the uMedSum filter (settings below) over the whole note, A/P included
B4	Namrata	0	B2's note, then one self-refine call: “review against the dialogue, list unsupported or missing items, rewrite”. No tools
Ours	Surya	up to 3	S/O verification → controller rules (below) → targeted questions → freeze → A/P synthesis → A/P verification → note → sign-off → patient summary

ID	Owner	Extra patient turns	After the checkpoint
C1	Surya	0	Ours with asking disabled: gaps left empty, unsupported claims deleted
C2	Surya	0	Ours with asking replaced by guessing: the 7B fills each gap from the conversation without asking
C3	Surya	exactly as many as ours used on that case and seed	Intake continues with its normal question policy for those turns, then C1's verification. Only **which** questions were asked differs from ours
C4	Surya	caps 1, 2, 3, 5	Ours with different question caps
B1	Surya	own intake	Single-agent intake → note. Context row only, not a matched branch

Controller rules for S/O claims (ours, C4; C1–C3 change only the ask step)

Verifier label	Condition	Action
supported	—	keep, with evidence
contradicted	confidence 0.8 or more, and a cited turn or measurement holds the correct value	correct from that evidence; re-verify
contradicted	otherwise	**ask** (S) or flag for doctor (O)
unsupported, or an empty mandatory field	Subjective; about a symptom, medication, allergy, duration or mandatory field	**ask** one targeted question
unsupported	Objective	flag for doctor ("needs examination")
unsupported	anything else	delete
after asking	patient confirms / denies / is unsure	keep with the new turn as evidence / delete / mark "patient unsure"
question cap reached	—	delete if unsupported; flag for doctor if contradicted

Questions are worded by the 7B from the claim and the attribute that failed, for example “What is the name and dose of your blood pressure tablet?”; the rules alone decide **whether** to ask.

RQ2 — all receive the same frozen A/P statements

On E1, the A/P statements come from ours′ synthesis and carry the features each one cites. On E2, they come from the draft note, and one shared citation step (the 7B lists supporting feature ids) runs once and is reused by every condition.

ID	Owner	Model	Rule	Rejects when
U1	Rahul	DeBERTa NLI	Entailment against all frozen features	Not kept under the uMedSum rule (B3 settings below)
U2	Namrata	Our verifier, S/O mode	Entailment against all frozen features	Label is not “supported”
U3	Rahul	Our verifier, A/P mode	Evidence chain, reference passage removed	chain_broken
Ours	Surya	Our verifier, A/P mode	Evidence chain plus reference passage	chain_broken

All four are scored on the **reject decision**: flagged (ours, U3) and deleted (U1, U2) both count as rejected, so ours cannot win just by never deleting. The effect of flag versus delete on the final note is reported separately.

B3 settings (uMedSum, published values)

Step	Setting
Premise	Frozen features of the encounter, as text
Split	PySBD sentences
Keep	entailment above 0.9
Remove	neutral + contradiction above 0.8
Uncertain	split into atomic facts with the 7B; keep facts with entailment above 0.5

Step	Setting
Missing information	top-2 key sentences from the features by MMR (MiniLM); missing if best similarity is 0.4 or less
Insert	at the position with the lowest GPT-2 perplexity

Supporting checks

ID	Owner	Change	Runs on
S1	Namrata	Verifier swapped: DeBERTa, prompted 7B, LoRA run 1, LoRA run 2	E2 claims
S2	Rahul	Verifier sees the whole dialogue instead of the top-k turns	E2 and E1 (ours)
S3	Surya	Single-agent intake checkpoint; then ours, C1, C2, C3	E1, 1 seed
S4	Namrata	Patient summary from the unverified record (checkpoint draft plus unverified A/P), no summary check	E1 (ours' encounters)

Guards (every condition)

The tool layer refuses, and logs as a blocked attempt: a write to features after the freeze; diagnosis or treatment wording sent to the patient before sign-off; finalising a note with an unresolved red flag; releasing a summary before sign-off. Items still flagged at sign-off are **excluded** from the patient summary.

Evaluation, statistics and annotation

Owner: Jyothika. **Reviewer:** Namrata. **Due:** known-answer metrics Oct 15; full scorer Oct 20; final numbers Oct 27.

Metrics

Group	Metric	Computed as	Source
RQ1 primary	Withheld-fact recovery	Withheld facts present **and** supported in the final note ÷ withheld facts	Patient fact ids + judge for presence

Group	Metric	Computed as	Source
RQ1 primary	S/O grounding rate	S/O claims supported by a patient turn or measurement ÷ all S/O claims	Verifier-independent judge, validated
RQ1 primary	Hallucination on gaps	Gap fills whose content is not in the case file ÷ gap fills	Case file
RQ1	Atomic and critical fact recall	Case facts (all / critical) in the final note ÷ case facts	Judge for presence
RQ1	Patient turns	Questions that reached the patient	Logs
RQ1 diagnostics	Gap-detection precision and recall; gaps resolved per question; redundant-question rate	Against case-file gaps at the checkpoint	Case file + logs
RQ2 primary	Valid-claim retention	Valid A/P statements not rejected ÷ valid A/P statements	E2 keys; reference A/P
RQ2 primary	Unsupported-A/P rejection	Precision and recall on the 30 planted A/P errors	E2 keys
RQ2 diagnostics	Per-step accuracy	Decomposition, citation step, verdict	Annotation
Patient reader	Unsupported statements per summary; next-step and warning-sign recall; Flesch–Kincaid grade	Against the signed note	Judge + textstat
Supporting	Verifier macro-F1 per error type; recall@k; citation precision; tool argument validity	—	Keys + logs
Safety	Unsafe-action rate; blocked attempts; missed red flags	Guard logs; 8 red-flag cases	Logs

Most primary metrics come from known answers. The LLM judge is used only to decide whether a fact is present in a note and whether a claim is supported, and it is never the same model as our verifier.

Judge prompt for fact presence

```
You are checking a clinical note against one fact.
FACT: {fact_text}
NOTE:
{note_text}
Is the fact stated in the note, word for word or with the same clinical
meaning?
Answer JSON only: {"present": true|false, "sentence": "<the note sentence, or
empty>"}
```

Statistics

```
import numpy as np
from scipy.stats import binomtest

def paired_bootstrap(a, b, n=1000, seed=0):
    """a, b: per-case scores (seeds already averaged), same case order."""
    rng = np.random.default_rng(seed); d = np.asarray(a) - np.asarray(b)
    boots = np.array([d[rng.integers(0, len(d), len(d))].mean() for _ in
range(n)])
    lo, hi = np.percentile(boots, [2.5, 97.5])
    p = 2 * min((boots <= 0).mean(), (boots >= 0).mean())
    return d.mean(), (lo, hi), min(p, 1.0)

def mcnemar_exact(x, y):
    """x, y: 0/1 recovered per (case, withheld fact, seed) for two
conditions."""
    b = sum(1 for i, j in zip(x, y) if i == 1 and j == 0)
    c = sum(1 for i, j in zip(x, y) if i == 0 and j == 1)
    return binomtest(b, b + c, 0.5).pvalue if b + c else 1.0

def holm(pvals, alpha=0.05):
    order = np.argsort(pvals); reject = [False] * len(pvals)
    for rank, i in enumerate(order):
        if pvals[i] > alpha / (len(pvals) - rank): break
        reject[i] = True
    return reject
```

Test families for Holm: RQ1 = ours against B3, C1, C2 and C3 on withheld-fact recovery, plus ours against B3 and C2 on grounding (6 tests). RQ2 = ours against U1 and U2 on valid-claim retention and on rejection recall (4 tests). Report every difference with its 95% interval, not only p-values.

Annotation (Sun Oct 25 to Mon Oct 26)

1. **Items:** 30 test notes (10 each from B2, B3 and ours) and 20 patient summaries (10 from ours, 10 from S4). System names removed; order shuffled; key kept in `annotation/key.csv` until all labels are in.
2. **Pairs:** Rahul + Jyothika and Surya + Namrata. Each item is labelled by both members of one pair, independently.
3. **Labels:** every S/O claim supported / contradicted / unsupported, with evidence turns; every A/P statement chain OK / broken; every summary sentence supported / unsupported; for 10 notes, presence of every case fact.
4. **Calibration first:** all four label the same 3 notes together and agree on borderline cases.
5. **Judge validation:** if judge–human Cohen's κ is 0.6 or more, judge-based metrics stay primary; otherwise they are reported as secondary and the conclusions rest on known-answer metrics.

Jyothika must be able to explain

- Why the bootstrap is paired, and what goes wrong if it is not
- Why McNemar counts only the discordant pairs
- What Holm controls, and what it costs in power
- What κ of 0.6 means, and why raw agreement is not enough
- Why 40 cases gives wide intervals, and how E2 compensates

Run matrix and budget ledger

Every run, in execution order. Token and GPU figures are planning estimates; replace them with measured values after the dev pilot on Oct 20. Checkpoint branching keeps cost low, because each intake is reused by every RQ1 condition.

Block	When	Conditions	Data	Seeds	Units	OpenRouter tokens	GPU-h
P0 Experiment 1 pilot	Oct 15	B2, B3, U1	Dev 10	1	10 encounters	~1M	0.5

Block	When	Conditions	Data	Seeds	Units	OpenRouter tokens	GPU-h
P1 Dev pilot, all conditions	Oct 18–20	All	Dev 10	1	10 checkpoints × all	~5M	2
V Verifier training	Oct 15–20	Run 1, run 2	Verifier train	—	2 runs	—	8–12 (see dry run)
R0 Intake	Oct 21	Multi-agent; single-agent	Test 40	2; 1	80 + 40 checkpoints	~10M	—
R1 RQ1 branches	Oct 22	B2, B3, B4, ours, C1, C2, C3, C4	Test 40	2	80 checkpoints × 11 runs	~18M	4
R2 RQ2 and verifier	Oct 22–23	U1, U2, U3, ours; S1	E2 120 + test A/P	1	120 drafts + 80 A/P sets	~1M	3
R3 Supporting checks	Oct 23–24	S2, S3, S4	Test 40, E2	1	~200 runs	~5M	2
R4 Judge	Oct 23–27	All outputs	—	—	~1,000 notes and summaries, batched per note	~6M (larger model)	—
Total						**~40M backbone + ~6M judge**	**~25**

Cost: the backbone at the listed OpenRouter price is about USD 5; the judge on a larger model is about USD 5–15. The hard cap is USD 50.

Ledger rules

1. Every run appends rows to `runs/ledger.csv` (format in Repository and shared data formats). No ledger row means the run did not happen.
2. Jyothika checks the ledger each evening during Phases 1 and 2 and posts spend so far.
3. If OpenRouter spend reaches **USD 35 before Oct 23**, apply the cut rules in order.
4. A run with errors in more than 2 encounters of a condition is re-run once for those encounters; the second failure is logged and reported.

Result-explanation playbook

The TA does not penalise negative results, only results we cannot explain. For each comparison we fix in advance what we expect, why, and which logged numbers will explain the outcome whichever way it goes. Every diagnostic below is already produced

by the runs, so explaining a result never needs a new experiment.

Comparison	Expected	Mechanism	If it holds, show	If it does not, check
Experiment 1: withheld facts at the checkpoint and after B3	Most still missing after B3	B3 compares the note with features that never contained the fact	Share missing before and after B3; B3 insertions that restore a real case fact	Withheld-fact leak rate (did intake ask anyway?); natural gaps without withholding
Experiment 1: U1 on valid A/P	Many valid statements rejected	Inferences are not entailed by symptoms	Rejection rate by A versus P; examples	Whether features already contain diagnosis words (copied from the reference note)
Ours vs. B3, withheld-fact recovery (H1a)	Ours higher	Gaps become questions while the patient is still present	Facts recovered in question turns; gaps resolved per question	Gap-detection recall (verifier missed gaps?); share of “not sure” answers; cap reached?
Ours vs. C1 (H1a)	Ours higher	Asking, not just checking earlier, supplies the evidence	Same as above	If equal: gaps were rarely detected, or answers rarely usable
Ours vs. C3 (H1b)	Ours higher at equal turns	Targeted questions address the exact gap	Per-question yield, ours vs. C3; topics of C3′s questions	If equal: the intake policy already asks about allergies and medications; check the overlap of C3′s questions with withheld topics

Comparison	Expected	Mechanism	If it holds, show	If it does not, check
Ours vs. C2, grounding (H1c)	C2 lower grounding, more hallucination on gaps	Guessing fills gaps with plausible defaults	Hallucination-on-gaps rate; examples like “no known allergies”	If C2 is equal: the 7B leaves gaps blank instead of guessing; report its abstention rate
Cap sweep (C4)	Recovery rises, then flattens	Each case has only two withheld facts plus a few natural gaps	Recovery and turns per cap	Gap count per case; redundant-question rate
Ours vs. U2, valid-claim retention (H2)	Ours higher	Same model, different rule: chain versus entailment	Retention by A versus P; same items rejected by U2 but kept by ours	If equal: the verifier in S/O mode is lenient on inferences; check U2′s verdicts on A/P
Ours vs. U1 and U2, rejection recall (H2)	Within 5 points	Planted A/P errors cite no supporting facts	Recall per error type	If lower: chain_broken under-predicted; check citation-step errors (did the 7B cite irrelevant facts?)
U3 vs. ours	Small difference	The evidence chain does most of the work; reference passages help some plans	Retention with and without passages; passage coverage	Low passage coverage explains no gain
S1, verifier comparison	LoRA ahead on negation and dose errors	Trained on exactly those perturbations	Per-type recall	Dev curves (overfitting?); spot-check label noise

Comparison	Expected	Mechanism	If it holds, show	If it does not, check
S3, single-agent intake	Larger RQ1 gain	A weaker intake leaves more gaps to recover	Gap count at the checkpoint, per intake	If smaller: the weaker intake also detects fewer topics to ask about
S4, unverified summary (H3)	More unsupported statements	The summary inherits note errors and flagged items	Unsupported statements traced to note versus added by the summary writer	If equal: the summary prompt is already conservative

Rule for writing results: every result sentence names the number, the mechanism and the diagnostic that supports it. For example: “Ours recovered X more withheld facts per encounter than B3 (95% CI …); Y% were disclosed in question turns, so the gain comes from asking.”

Viva preparation

The TA expects every member to know every part and to explain why each result happened. Any member may be asked about any component.

Everyone must be able to explain, without notes

1. The two failures (unrecoverable gaps; fact–inference category error) in two sentences each, with the chest-pain example.
2. One encounter end to end: intake, checkpoint, S/O verification, question, freeze, A/P verification, doctor note, patient summary.
3. Which parts are core (agentic control, reasoning) and which are supporting (tools, retrieval, fine-tuning), and why multi-agent intake is held fixed.
4. What B, C, U and S conditions are, and what each pair comparison isolates (ours vs. C3, ours vs. U2).
5. The main result for each research question, its number, and the diagnostic that explains it.
6. One failure case and the stage where it went wrong.

Question bank (owner writes the answer in docs/viva_questions.md; all read it)

Question	Owner
Why not just run uMedSum on Aegle's note, as B3 does?	Surya
Why does every condition start from the same checkpoint, and what would go wrong otherwise?	Surya
Why does C3 get exactly as many turns as ours on each case?	Surya
Who decides to ask: the verifier, the controller or the LLM?	Surya
How does the system avoid giving medical advice to the patient?	Surya
Why top-k turns, and how was k chosen?	Rahul
Where do the reference passages come from, and why not clinical guidelines?	Rahul
How does B3 decide information is missing, and why can it not recover a withheld fact?	Rahul
What is the difference between U1 and U2, and why do you need both?	Rahul
Is the verifier one model or two? What changes between S/O and A/P mode?	Namrata
Why fp32 compute and 4-bit weights on the 1080 Ti?	Namrata
The training data is synthetic; why should results transfer?	Namrata
How is the verifier's confidence computed, and what is it used for?	Namrata
Why is RQ2 scored on the reject decision rather than on the final note?	Jyothika
Why a paired bootstrap, and what does the interval mean?	Jyothika
How did you validate the LLM judge, and what if κ had been low?	Jyothika

Question	Owner
40 encounters is small; how confident are you?	Jyothika
If ours had lost to C3, what would you conclude?	All
What would you do with another month?	All

Sessions

Date	Session	Format
Tue Oct 20, evening	Components	Each owner explains their code for 15 minutes; the reviewer explains it back in 5
Thu Oct 29	Results	Each owner presents results with the playbook diagnostics; the others challenge them
Fri Oct 30	Mock viva	45 minutes; each member answers 4 questions about components they did not build; weak answers rewritten the same day

Rule for coding assistants: code written with a coding assistant is allowed, but nothing is merged until its owner can explain every function to the reviewer, who signs off in the pull request description.

Risks, cut rules and final checklist

Risks

Risk	Early sign	Response	Owner
QLoRA too slow or fails on the 1080 Ti	Dry run on Oct 13	Apply the dry-run decision rule: move training to a Kaggle or Colab T4	Namrata
Withheld facts leak to general questions	Audit on Oct 15 shows more than 2 leaks in 20	Tighten rule 3 of the patient prompt; score recovery only on facts still missing at the checkpoint	Surya

Risk	Early sign	Response	Owner
Simulated patient invents facts	Any invented fact in the audit	Lower temperature; stricter prompt; drop cases that keep failing	Surya
Verifier rarely detects gaps, so asking rarely triggers	Gap-detection recall below 0.5 on dev (Oct 20)	Add an empty-mandatory-field check that needs no verifier; also report a run with oracle gaps from the case file, to separate detection from asking	Surya, Namrata
Aegle-style intake unstable on the 7B	B2 failing on dev by Oct 16	Simplify to one orchestrator and two fixed specialists; it is a fixed substrate, so this does not affect the research questions	Surya
Reference passages missing for many topics	Coverage below 60% on dev	U3 (evidence chain alone) becomes the main RQ2 condition; report coverage	Rahul
OpenRouter rate limits or outages	Errors during Phase 2	Retry with backoff; run blocks overnight; keep a second provider key ready	Surya
TA feedback asks for structural changes	Feedback on Oct 14–15	Feedback-gate procedure; start-now work is reused	All
Limited annotation time	Behind on Oct 26	Reduce to 20 notes; rely on known-answer metrics for primary claims	Jyothika

Cut rules (apply in this order, only as needed)

1. C4: caps 1 and 3 only.
2. B4 on dev only, reported as a supplementary row.
3. S2 on E2 only.
4. S3 on 20 encounters.

5. Second seed for B4, C2 and C4 dropped (kept for B2, B3, ours, C1 and C3).

Never cut: B3, C1, C2, C3, U1, U2, Experiment 1, judge validation, or the mock viva.

Final checklist (Sat Oct 31)

- [ ] Feedback logged in `docs/ta_feedback.md`, with every point addressed
- [ ] Experiment 1 result shows the size of both failures
- [ ] RQ1 and RQ2 results with 95% intervals and Holm-corrected tests
- [ ] Every result sentence names its mechanism and diagnostic
- [ ] Supporting checks S1–S4 reported, with no claims beyond them
- [ ] Failure analysis of 30 errors by stage
- [ ] Judge validation (κ) reported
- [ ] Demo: one encounter showing a question recovering a withheld fact, the doctor note with citations and flags, and the patient summary
- [ ] Repository with README, configs, run scripts, `runs/ledger.csv` and the frozen commit hash
- [ ] `docs/viva_questions.md` complete; mock viva held
- [ ] Report compiled in the ACL template and submitted